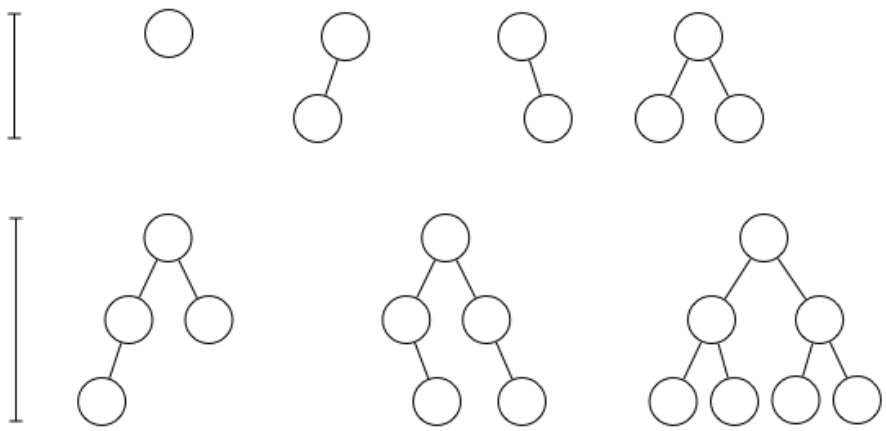
COMP9024 19T3 Data Structures and Algorithms

Week 7 Problem Set
Binary Search Trees

1. (Tree properties)

   a. Derive a formula for the minimum height of a binary search tree (BST) containing *n* nodes. Recall that the height is defined as the number of edges on a longest path from the root to a leaf. You might find it useful to start by considering the characteristics of a tree which has minimum height. The following diagram may help:

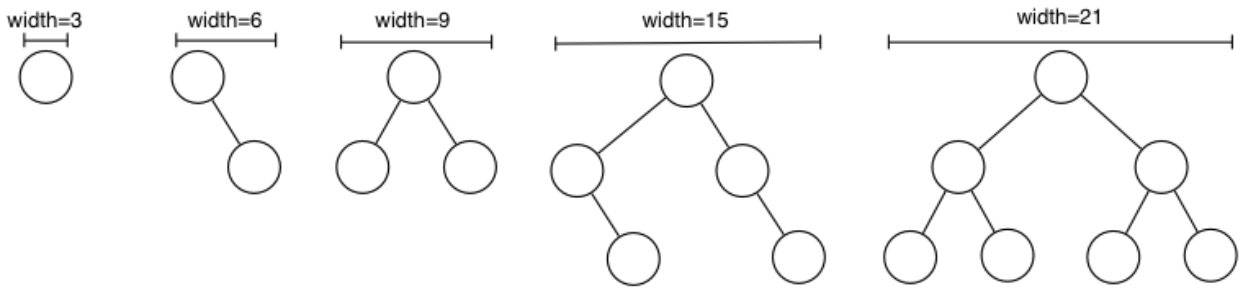


   b. In the Binary Search Tree ADT (`BSTree.h`, `BSTree.c`) from the lecture, implement the function:

```
int TreeHeight(Tree t) { ... }
```

   to compute the height of a tree.

   c. Computing the height/depth of trees is useful for estimating their search efficiency. For *drawing* trees, we're more interested in their *width*. For some simple trees the following diagrams show a useful definition of tree width if you want to keep reasonable spacing:



      i. Derive a formula for the width of a tree that generalises from the examples.

      ii. Add a new function to the `BSTree` ADT which computes the width of a tree. Use the following function interface:

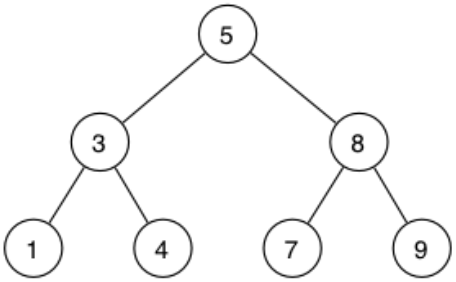
```
int TreeWidth(Tree t) { ... }
```

We have created a script that can automatically test your program. To run this test you can execute the `dryrun` *program that corresponds to this exercise. It expects to find the program named* `BSTree.c` *with your implementation for* `TreeHeight()` *and* `TreeWidth()` *in the current directory. You can use dryrun as follows:*

```
prompt$ 9024 dryrun BSTree
```

2. (Tree traversal)

   Consider the following tree and its nodes displayed in different output orderings:



Infix Order	1 3 4 5 7 8 9
Prefix Order	5 3 1 4 8 7 9
Postfix Order	1 4 3 7 9 8 5
Level Order	5 3 8 1 4 7 9

a. What kind of trees have the property that their infix output is the same as their prefix output? Are there any kinds of trees for which all four output orders will be the same?

b. Design a recursive algorithm for prefix-, infix-, and postfix-order traversal of a binary search tree. Use pseudocode, and define a single function `TreeTraversal(tree,style)`, where `style` can be any of `"NLR"`, `"LNR"` or `"LRN"`.

3. (Insertion and deletion)

a. Show the BST that results from inserting the following values into an empty tree in the order given:

```
6 2 4 10 12 8 1
```

b. Let `t` be your answer to question a., and consider executing the following sequence of operations:

```
TreeDelete(t,12);
TreeDelete(t,6);
TreeDelete(t,2);
TreeDelete(t,4);
```

Assume that deletion is handled by joining the two subtrees of the deleted node if it has two child nodes. Show the tree after each delete operation.

4. **Challenge Exercise**

The function `showTree()` from the lecture displays a given BST sideways. A more attractive output would be to print a tree properly from the root down to the leaves. Design and implement a new function `showTree(Tree t)` for the Binary Search Tree ADT (`BSTree.h`, `BSTree.c`) to achieve this.

Please email me your solution. The best solution will be added to our BST ADT implementation and used in the next lecture (week 8). Both the attractiveness of the visualisation and the simplicity of the code will be judged.

Assessment

After you've solved the exercises, go to COMP9024 19T3 Quiz Week 7 to answer 4 quiz questions on this week's problem set (Exercises 1-3 only) and lecture.

The quiz is worth 2 marks.

The deadline for submitting your quiz answers is **Monday, 4 November 11:00:00am**.

A reminder of the **quiz rules**:

Do …

- use your own best judgement to understand & solve a question
- discuss quizzes on the forum only **after** the deadline on Monday

Do not …

- post specific questions about the quiz **before** the Monday deadline
- agonise too much about a question that you find too difficult